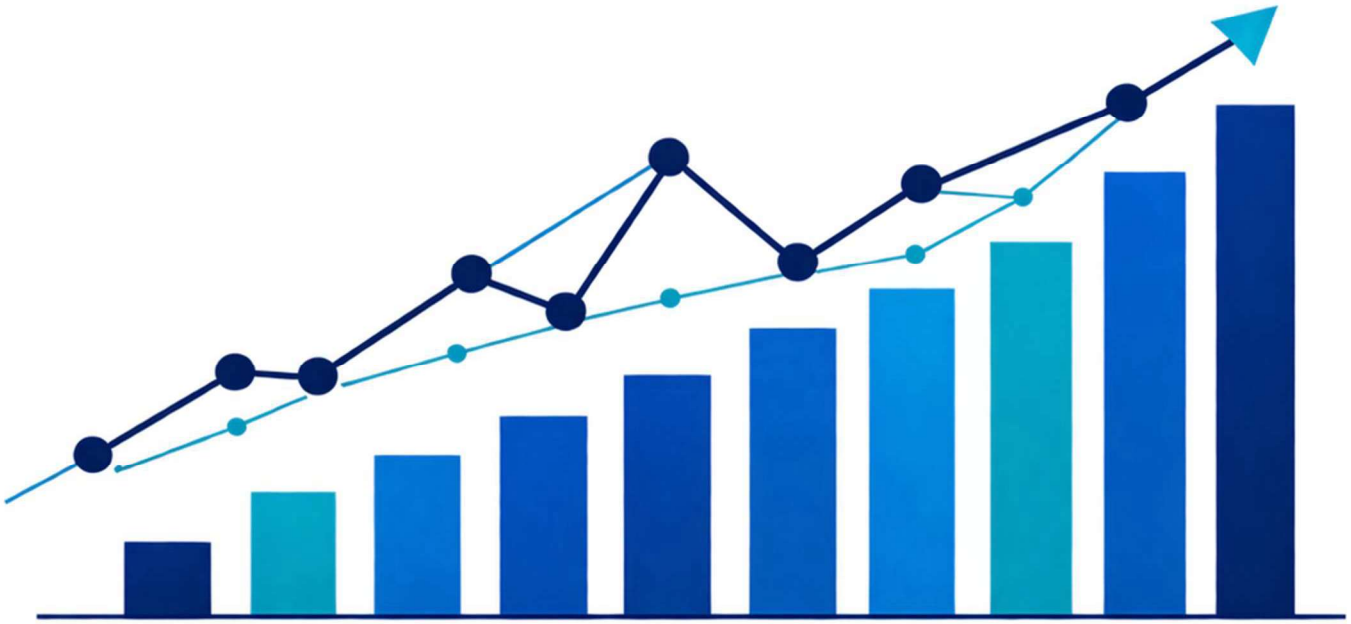
Learnlytics .handbook

Business Intelligence & Dashboards For Data Analyst

Concise | Comprehensive | Career-Defining



LEARN SMART.
ANALYZE BETTER.
GROW FASTER.

Business Intelligence & Dashboards

A Complete 10-Chapter Practice Workbook

First Edition

Published by Learnlytics.handbook

www.learnlyticshandbook.shop | learnlytic.handbook@gmail.com

Copyright © 2026 Learnlytics.handbook. All rights reserved.

No part of this publication may be reproduced, distributed, stored in a retrieval system, or transmitted in any form or by any means — electronic, mechanical, photocopying, recording, or otherwise — without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other non-commercial uses permitted by copyright law.

For permission requests or licensing inquiries, write to the publisher at the address above.

ISBN: 978-1-2565-5895-9 (Digital Edition)

First published: 2026

Printed and distributed digitally worldwide.

IMPORTANT: LICENSE & COPYRIGHT NOTICE This document and all included materials are the exclusive intellectual property of Learnlytics.handbook. Your purchase grants you a single-user, personal license. This file is digitally tracked. Unauthorized reproduction, forwarding, resale, or distribution of this material — whether in private groups, Telegram channels, or public forums — is strictly prohibited and constitutes copyright infringement. Our legal team actively monitors digital distribution networks. Violations of this agreement will result in immediate termination of your access, notification to your educational/professional institution, and formal legal action under applicable international and domestic copyright laws. Thank you for respecting the hard work that went into creating this guide!

Disclaimer: The information in this guide is provided for educational purposes. BI tool features and cloud platform capabilities evolve rapidly. Always refer to the official documentation of your chosen BI platform for the most current specifications. The author and publisher make no warranty, express or implied, with respect to the material contained herein.

Typeset in Arial | Cover design: Learnlytics.handbook

Made in India with for Data Analysts Worldwide

Table of Contents

10 Units • 200 Questions • A Complete BI Journey

Unit 1 Dashboard Design & UX Strategy ..... 04 – 19

Unit 2 BI Data Modeling (The Star Schema)..... 20 – 41

TRACK A: Power BI (The Microsoft Giant)

Unit 3 Power BI Desktop Fundamentals ..... 42 – 64

Unit 4 DAX (Data Analysis Expressions) ..... 65 – 90

Unit 5 Power BI Service & Governance .....91 – 118

TRACK B: Tableau (The Visual Standard)

Unit 6 Tableau Basics & Data Architecture .....119 – 147

Unit 7 Advanced Tableau (LODs & Interactivity).....148 – 179

TRACK C: Specialised & Enterprise BI

Unit 8 Looker Studio (Lightweight / Marketing BI).....180 – 215

Note: This is usually for "Senior" or "Product" Data Analysts.

Unit 9 Looker & LookML (Modern Enterprise BI) .....216 – 258

Unit 10 KPI Tracking & The Metrics Layer.....259 – 296

INTERVIEW-BASE UNIT TEST

Unit 1: Dashboard Design & UX Strategy

Business Intelligence & Dashboards

LEVEL 1: WARMUP TEST — 5 MCQ Questions

Instructions: Choose the single best answer for each question.

Q1. Which of the following is a LEAD indicator?

A) Total revenue earned last quarter
B) Number of sales calls made this week ✓
C) Annual profit margin
D) Customer churn rate last month

Q2. What does the "5-Second Rule" in dashboard design state?

A) A dashboard must load in 5 seconds
B) Users should click a chart within 5 seconds
C) The most important insight should be understood within 5 seconds of viewing ✓
D) Tooltips should appear within 5 seconds of hovering

Q3. Which Gestalt Principle states that elements enclosed within a border are perceived as a group?

A) Proximity
B) Similarity
C) Continuity
D) Enclosure ✓

Q4. Which chart type is MOST appropriate to show how revenue changed month by month over a year?

A) Pie Chart
B) Scatter Plot
C) Line Chart ✓
D) Heatmap

Q5. What is the standard format for writing a User Story in dashboard design?

A) "The dashboard should show [metric] for [user]."
B) "As a [type of user], I want to [action], so that I can [goal]." ✓
C) "When [condition], display [chart] to [user]."
D) "Build [feature] for [user] because [reason]."

🎯 LEVEL 2: INTERVIEWER TEST — 15 Intermediate Semi-Advance Questions

Answer all questions in a conversational, thoughtful student format — acknowledge the "why" behind concepts, keep tone practical, and include brief code examples where relevant.

📝 Section A: Descriptive Questions (4 Questions)

Q1. Explain the difference between Lead and Lag Indicators. Why is it important to include BOTH types on a business intelligence dashboard? Provide examples from two different industries.

☑ Answer

Great question — this is something I find really intuitive once you frame it correctly.

Lead indicators are forward-looking metrics you can act on RIGHT NOW. They give early warning signals before an outcome is confirmed. For example, the number of sales calls made today is a lead indicator — you can increase calls to impact future revenue.

Lag indicators are backward-looking results that confirm what already happened. Total revenue earned last quarter is a lag — it tells you the outcome, but you cannot change it.

The reason BOTH matter on a dashboard is this: if you only show lag indicators, you are always reacting — like reading yesterday's newspaper to make today's decisions. If you only show lead indicators, you don't know whether your efforts actually worked.

Industry Example 1 (Healthcare): Lead = Patient wait time today (you can assign more staff now). Lag = Monthly patient satisfaction score (confirms whether efforts worked).
Industry Example 2 (Marketing): Lead = Email open rate this campaign (signal of engagement). Lag = Customer acquisition cost last quarter (confirms campaign ROI).

A smart dashboard always pairs a predictive metric with its confirmation metric — so decision-makers can both anticipate and verify performance.

Q2. Describe all five Gestalt Principles relevant to dashboard design. For each principle, explain how misapplying it can negatively affect user experience.

Answer

Gestalt principles are fascinating because they describe how our brains are wired — we perceive patterns before we read text. Understanding these is fundamental to design that feels intuitive.

1. Proximity: Elements placed close together are seen as a group. Misapplication: Scattering related charts randomly confuses users about which metrics belong together — they spend mental energy figuring out structure instead of reading data.
2. Similarity: Things that look alike (same color/shape) are seen as related. Misapplication: Using random colors for each chart destroys meaning — users cannot identify which metrics share a category.
3. Enclosure: Elements within a border/box are perceived as a group. Misapplication: Placing unrelated charts side-by-side without visual separation creates false relationships — users assume they are linked.
4. Continuity: The eye follows smooth, continuous lines naturally. Misapplication: Inconsistent row/column alignment forces the eye to jump unnaturally, increasing cognitive fatigue.
5. Figure/Ground: The brain separates objects from their background. Misapplication: A dark or busy background competes with data points — the "figure" (data) doesn't pop, reducing readability.

Well-designed dashboards apply multiple principles simultaneously — this is why great design feels effortless while poor design feels exhausting.

Q3. What is Color Theory in the context of dashboard design? Explain the three roles of color (Categorical, Sequential, Diverging) with real-world dashboard examples for each.

Answer

Color theory in dashboards is really about discipline — using color with purpose rather than decoration. The golden rule I always keep in mind: if everything is highlighted, nothing is highlighted.

1. Categorical Color: Used to distinguish between different groups. Example: A retail dashboard uses blue for Product A revenue bars and orange for Product B revenue bars — different colors, different categories. Maximum 5-7 distinct colors; beyond that, viewers cannot keep track.
2. Sequential Color: Used to show intensity or magnitude through light-to-dark shading. Example: A supply chain heatmap where light yellow = 1 day delayed, medium orange = 3 days, dark red = 5+ days delayed. The shade itself communicates severity without a legend.
3. Diverging Color: Used to show deviation from a center point. Example: A financial dashboard showing sales vs. target — dark green for above target, white/grey for at target, dark red for below target. This immediately communicates performance direction.

Additional consideration: 8% of men have color vision deficiency, so blue-orange is a safer distinguishing pair than red-green. Accessibility is not optional in professional dashboard design.

Q4. Explain the concept of "Request Bloat" in dashboard design. What causes it, and what professional techniques can an analyst use to say no while maintaining stakeholder relationships?

Answer

Request bloat is something every analyst eventually faces — it's when stakeholders keep adding charts, metrics, and features until the dashboard collapses under its own complexity. A dashboard with 30 charts tells no story.

Why it happens: The HiPPO effect (Highest Paid Person's Opinion), FOMO from departments worried their data won't be visible, no clear ownership of the dashboard, and "just in case" thinking where metrics are added speculatively.

The 3-test filter I use for every requested chart:
- Decision Test: What decision does this chart enable? No clear answer → reject.
- Audience Test: Which user story does this serve? Not in scope → reject.
- Frequency Test: How often will someone look at this? Rarely → move to an appendix page.

Professional techniques for saying no:
- "Not Now" approach: "That's a great idea for Version 2 — let's validate the core first."
- Appendix approach: "We'll put this on Page 3 for analysts — keeps the main dashboard clean."
- Evidence approach: "After the first month, we'll review usage analytics and revisit this."

The key insight is that saying no professionally actually builds credibility — stakeholders learn to trust that what IS on the dashboard truly matters.

Section B: Scenario-Based Descriptive Questions (4 Questions)

Q1. SCENARIO: A hospital's operations team asks you to build a single dashboard for three users: the Hospital Director (needs bed occupancy % at a glance), Ward Nurses (need today's patient list by ward), and a Health Data Analyst (needs historical admission patterns with filters). How would you approach this, and what interactivity features would you use?

Answer

This is a classic multi-audience problem, and the answer is that one page cannot serve all three users effectively. Here's how I'd approach it:

User Story Strategy: I'd write three formal user stories first — one for each persona — to define exactly what each user needs before designing anything.

Page structure using interactivity:

• Page 1 — Executive View: Hospital Director. KPI card showing Bed Occupancy % (large, top-center). RAG (Red/Amber/Green) color coding. Updated daily. No filters — just the key number with a trend sparkline.
• Page 2 — Operational View: Ward Nurses. Table showing today's patient list. Slicer to filter by Ward. Cross-filtering between ward map and patient list. Refreshed every 4 hours or in near-real-time.
• Page 3 — Analytical View: Data Analyst. Full historical admission data with date range slicers, department filters, diagnosis filters. Drill-down from ward level → individual patient admission records. Export capability.

Navigation Design: Clear page tabs or a home page with three role-based buttons: "Hospital Director View", "Ward Operations", "Analyst Explorer". Each button uses bookmarks/page navigation to route the right person to the right page.

The interactivity features I'd prioritize: drill-downs (ward → patient), slicers (department, date), cross-filtering, and tooltips showing bed status, days occupied, and ward capacity. This turns one dataset into three purpose-built experiences.

Q2. SCENARIO: You have designed a clean 8-chart sales dashboard. A senior VP storms into your meeting and demands you add 7 more charts immediately — including a map of revenue by country for all 50 countries, hourly revenue fluctuations, and a chart showing revenue in 23 different currencies. How do you respond?

Answer

This is the HiPPO (Highest Paid Person's Opinion) problem in action — and it's a situation that tests both technical judgment and communication skills. My response would be structured, not defensive.

First, I'd acknowledge the intent: "These are genuinely interesting data points — let me think through where each of these fits best for your needs."

Then I'd apply the 3-filter test openly:
• Revenue map across 50 countries: "A 50-country map at this scale is visually unreadable — dots overlap and the chart loses meaning. Instead, I'd suggest a top 10 countries bar chart, with a 'View All' drill-down page for the full breakdown. This gives you the headline insight instantly."
• Hourly revenue fluctuations: "This serves our real-time operations team — I'd recommend this on a separate operations sub-page or a dedicated live monitoring dashboard. It would overwhelm an executive sales view."
• Revenue in 23 currencies: "This is an analyst-level requirement used by 1-2 finance team members. I'd build this as a linked report accessible from the dashboard, not on the main dashboard page itself."

The framing I'd use with the VP: "My goal is to make sure you can read the dashboard in 5 seconds and trust every number on it. Adding 7 more charts will make that impossible — but I can build a connected Page 2 that houses all this detail for your team."

This protects the dashboard's integrity while showing the VP their needs will be met — just in the right place.

Q3. SCENARIO: An e-commerce analyst is building a campaign performance dashboard. The marketing team wants to compare 5 campaigns' total clicks, show how daily website traffic changed over 30 days, AND show what % of revenue came from each of 4 product categories. Which chart types would you use for each, and why?

Answer

This is a classic chart selection problem — three different "questions" requiring three different chart types.

Requirement 1 — Compare 5 campaigns' total clicks: Use a HORIZONTAL BAR CHART. Why? This is a comparison question (which is bigger/smaller?). Horizontal bars are ideal when you have 5+ categories because labels fit naturally on the left axis without rotation. The eye compares bar lengths intuitively — much cleaner than a vertical bar for many categories.

Requirement 2 — How daily traffic changed over 30 days: Use a LINE CHART. Why? This is a trend/time-series question. Line charts are the right tool when you want to show direction, momentum, and change over time. The connecting line between daily points helps the eye follow the trajectory — a bar chart for 30 days looks cluttered and doesn't communicate trend as clearly.

Requirement 3 — Revenue % from 4 product categories: Use a PIE CHART or DONUT CHART (4 slices is within the maximum of 5). Why? This is a composition question (part-to-whole). With only 4 categories, slices are large enough to be distinguishable. I'd choose a donut chart specifically — it provides a clean center space for the total revenue figure, adding extra context.

If the team also wanted to compare the category revenue split across months, I'd upgrade from pie to a STACKED BAR CHART — it handles both composition AND comparison across time periods simultaneously.

Q4. SCENARIO: You are reviewing a colleague's dashboard and notice: a 3D pie chart with 9 slices, a dual-axis chart mixing revenue and profit %, and a bar chart with a Y-axis starting at 850,000 instead of 0. Identify all design violations, explain why each is problematic, and propose the correct solution.

Answer

This is a textbook example of three separate anti-patterns — I'd address each one clearly in a design review.

Violation 1 — 3D Pie Chart with 9 slices: Problem: 3D perspective distorts proportions — slices at the "front" of the chart appear larger than those at the "back", even if they have the same value. This actively misleads viewers. 9 slices is also far beyond the maximum of 5 — human eyes cannot

distinguish tiny slices reliably. Solution: Replace with a horizontal bar chart sorted by value (largest to smallest). This handles unlimited categories and allows precise length comparison.

Violation 2 — Dual-axis chart (Revenue + Profit %): Problem: Two Y-axes on one chart create cognitive overload. Viewers must constantly look back and forth to understand which axis corresponds to which line, and the visual relationship between the two lines is misleading — small changes on one axis look equivalent to large changes on the other. Solution: Split into two separate single-axis charts placed side by side. Viewers can compare them contextually without confusion.

Violation 3 — Y-axis starting at 850,000 (truncated): Problem: A truncated Y-axis dramatically exaggerates differences. What might be a 2% variance in revenue appears as a massive cliff on screen, manufacturing urgency that doesn't exist in the data. This is a data integrity issue, not just an aesthetic one. Solution: Set the Y-axis to start at 0, unless there is a documented, explicit reason to truncate (and even then, clearly label the truncation with a break symbol on the axis).

I'd communicate this to my colleague as: "Three quick fixes that will make this dashboard more trustworthy — let me show you the corrected versions and the reasoning behind each."

💻 Section C: Coding Questions (4 Questions)

Q1. Using Python and pandas, write code to calculate both a Lead KPI (7-day rolling average of daily active users) and a Lag KPI (monthly total revenue). The dataset has columns: date, daily_active_users, and revenue.

☑ Answer

Understanding the "why" here matters: rolling averages smooth out daily noise so you can see the trend signal clearly — that's what makes it useful as a lead indicator. Monthly total is a clean lag confirmation.

Code:

```
import pandas as pd

# Sample dataset
data = {
 "date": pd.date_range(start="2024-01-01", periods=90, freq="D"),
 "daily_active_users": [200 + i*2 for i in range(90)],
 "revenue": [5000 + i*100 for i in range(90)]
}
df = pd.DataFrame(data)
df["date"] = pd.to_datetime(df["date"])

# ── LEAD INDICATOR: 7-Day Rolling Average of Daily Active Users ──
df["rolling_avg_dau"] = df["daily_active_users"].rolling(window=7).mean()
```

```
# ── LAG INDICATOR: Monthly Total Revenue ──
df["month"] = df["date"].dt.to_period("M")
monthly_revenue = df.groupby("month")["revenue"].sum().reset_index()
monthly_revenue.columns = ["month", "total_monthly_revenue"]

# ── Output ──
print("Lead Indicator — 7-Day Rolling Avg DAU:")
print(df[["date", "daily_active_users", "rolling_avg_dau"]].tail(10))

print("\nLag Indicator — Monthly Total Revenue:")
print(monthly_revenue)
```

Q2. Write a Python function that evaluates a KPI value against a target and returns a color-coded RAG (Red/Amber/Green) status. The function should accept: value, target, and an amber_threshold (default 10%). Apply this to a list of 5 KPIs and print a styled summary.

Answer

The RAG pattern is foundational to dashboard color theory — it implements the diverging color role (green = above, amber = warning, red = below). The threshold logic is where the real value is.

Code:

```
def rag_status(value, target, amber_threshold=0.10):
 """
 Returns RAG status for a KPI.
 value: actual KPI value
 target: expected target value
 amber_threshold: % below target before Amber fires (default 10%)
 """
 if target == 0:
 return "GREY", "No target defined"

 performance_ratio = value / target

 if performance_ratio >= 1.0:
 return "GREEN", f"{value} ≥ target {target} ({performance_ratio*100:.1f}%)"
 elif performance_ratio >= (1 - amber_threshold):
 return "AMBER", f"{value} within {amber_threshold*100:.0f}% of target
{target}"
 else:
 return "RED", f"{value} significantly below target {target}"



# ── Apply to 5 KPIs ──
kpis = [
 {"name": "Monthly Revenue", "value": 120000, "target": 100000},
 {"name": "Customer Satisfaction", "value": 88, "target": 90},
 {"name": "Sales Conversion Rate", "value": 4.2, "target": 5.0},
```

```
 {"name": "NPS Score", "value": 72, "target": 70},
 {"name": "Churn Rate (lower=good)", "value": 6.5, "target": 5.0},
]

print(f"{"KPI":<30} {"VALUE":>10} {"TARGET":>10} {"STATUS":<8} DETAIL")
print("-" * 80)
for kpi in kpis:
 status, detail = rag_status(kpi["value"], kpi["target"])
 icon = "☑" if status == "GREEN" else "⚠" if status == "AMBER" else "❌"
 print(f"{kpi["name"]:<30} {str(kpi["value"]):>10} {str(kpi["target"]):>10}
{icon} {status:<6} {detail}")
```

Q3. Using Python and matplotlib, write code to create a properly designed dashboard chart panel that demonstrates chart selection best practices: a bar chart for comparison, a line chart for trends, and a donut chart for composition — all in one figure with correct formatting.

☑ Answer

Chart selection best practice says: match the chart to the relationship being shown. This code demonstrates all three categories in one figure.

Code:

```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import numpy as np

fig, axes = plt.subplots(1, 3, figsize=(16, 5))
fig.suptitle("Dashboard Chart Selection Best Practices", fontsize=14,
fontweight="bold")

# ── Chart 1: BAR CHART — Comparison ──
campaigns = ["Email", "Paid Search", "Social", "Referral", "Direct"]
clicks = [12000, 9500, 7800, 4300, 3100]
colors = ["#2E75B6" if c == max(clicks) else "#8CB4DC" for c in clicks]
axes[0].barh(campaigns, clicks, color=colors)
axes[0].set_title("Campaign Clicks (Comparison)", fontweight="bold")
axes[0].set_xlabel("Total Clicks")
axes[0].invert_yaxis() # Largest bar at top
axes[0].spines[["top","right"]].set_visible(False)

# ── Chart 2: LINE CHART — Trend/Time-Series ──
days = list(range(1, 31))
traffic = [1200 + np.random.randint(-100, 200) + d*20 for d in days]
axes[1].plot(days, traffic, color="#2E75B6", linewidth=2, marker="o",
markersize=3)
axes[1].fill_between(days, traffic, alpha=0.1, color="#2E75B6")
axes[1].set_title("Daily Website Traffic — 30 Days (Trend)",
fontweight="bold")
axes[1].set_xlabel("Day")
```

```
axes[1].set_ylabel("Visitors")
axes[1].spines[["top","right"]].set_visible(False)

# ── Chart 3: DONUT CHART — Composition ──
categories = ["Electronics", "Clothing", "Home", "Sports"]
revenue_pct = [38, 27, 21, 14]
palette = ["#2E75B6", "#F4A261", "#2A9D8F", "#E76F51"]
wedges, texts, autotexts = axes[2].pie(
 revenue_pct, labels=categories, colors=palette,
 autopct="%1.0f%%", startangle=90,
 wedgeprops=dict(width=0.55) # Donut hole
)
axes[2].set_title("Revenue by Category (Composition)", fontweight="bold")

plt.tight_layout()
plt.savefig("dashboard_charts.png", dpi=150, bbox_inches="tight")
plt.show()
print("Chart saved as dashboard_charts.png")
```

Q4. Write a Python function called apply_5_second_rule() that takes a list of KPI dictionaries (with name, value, target, and priority fields) and sorts them by priority, flags any that fail a simple threshold test, and returns a formatted layout plan showing Zone 1 (top-left), Zone 2 (top-right), and Zone 3 (bottom) assignments.

Answer

This is the 5-Second Rule and Information Hierarchy implemented in code — the most critical KPI gets Zone 1 automatically based on priority and performance status.

Code:

```
def apply_5_second_rule(kpis):
 """
 Takes a list of KPI dicts and assigns them to dashboard zones
 based on the 5-Second Rule (most critical info gets Zone 1).
 Each KPI dict: {name, value, target, priority (1=highest)}
 """
 def rag(v, t):
 ratio = v / t if t else 0
 if ratio >= 1.0: return "GREEN"
 elif ratio >= 0.90: return "AMBER"
 return "RED"

 # Annotate each KPI with RAG and sort by priority
 for k in kpis:
 k["rag"] = rag(k["value"], k["target"])
 k["pct"] = round((k["value"] / k["target"]) * 100, 1) if k["target"] else 0

 # Sort by priority (1 = most important), then RED first within same priority
 rag_order = {"RED": 0, "AMBER": 1, "GREEN": 2}
```

```
 sorted_kpis = sorted(kpis, key=lambda k: (k["priority"],
rag_order[k["rag"]]))

 # Zone assignment
 zones = {"Zone 1 (Top-Left — PRIMARY)": [],
 "Zone 2 (Top-Right — SECONDARY)": [],
 "Zone 3 (Bottom — TERTIARY)": []}

 for i, kpi in enumerate(sorted_kpis):
 if i == 0:
 zones["Zone 1 (Top-Left — PRIMARY)"].append(kpi)
 elif i <= 2:
 zones["Zone 2 (Top-Right — SECONDARY)"].append(kpi)
 else:
 zones["Zone 3 (Bottom — TERTIARY)"].append(kpi)

 # Print layout plan
 print("=" * 60)
 print("DASHBOARD LAYOUT PLAN — 5-Second Rule Applied")
 print("=" * 60)
 for zone, items in zones.items():
 print(f"\n{zone}:")
 for k in items:
 flag = "🔴" if k["rag"]=="RED" else "🟡" if k["rag"]=="AMBER" else "🟢"
 print(f" {flag} {k["name"]}: {k["value"]} / {k["target"]} ({k["pct"]}% of
target)")
 return zones



# ── Test ──
kpis = [
 {"name": "Total Revenue", "value": 95000, "target": 100000, "priority": 1},
 {"name": "Customer NPS", "value": 72, "target": 75, "priority": 2},
 {"name": "Churn Rate", "value": 6.0, "target": 5.0, "priority": 2},
 {"name": "New Signups", "value": 430, "target": 400, "priority": 3},
 {"name": "Support Tickets", "value": 210, "target": 150, "priority": 3},
]
apply_5_second_rule(kpis)
```

Section D: Database Scenario-Based Questions (3 Questions)

Q1. DATABASE SCENARIO: You have a sales_transactions table with columns: transaction_id, salesperson_id, region, sale_date, revenue, and target_revenue. Write SQL to generate a KPI summary report showing: total actual revenue, total target revenue, attainment %, and RAG status (Green ≥ 100%, Amber ≥ 90%, Red < 90%) — grouped by region. Sort by attainment ascending (worst-performing regions first).

Answer

This is a classic dashboard backend query — grouping real vs. target data and applying business logic (RAG classification) inside SQL. Sorting worst-first is intentional: it ensures problem regions are visible immediately on the dashboard.

SQL:

```
SELECT
 region,
 SUM(revenue) AS total_actual_revenue,
 SUM(target_revenue) AS total_target_revenue,
 ROUND(
 SUM(revenue) * 100.0 / NULLIF(SUM(target_revenue), 0),
 2) AS attainment_pct,
 CASE
 WHEN SUM(revenue) >= SUM(target_revenue)
 THEN 'GREEN — On/Above Target'
 WHEN SUM(revenue) >= SUM(target_revenue) * 0.90
 THEN 'AMBER — Within 10% of Target'
 ELSE
 'RED — Below 90% of Target'
 END AS rag_status
FROM
 sales_transactions
GROUP BY
 region
ORDER BY
 attainment_pct ASC; -- Worst performers first (critical for dashboard design)

-- NULLIF prevents division-by-zero when target is 0
-- Multiply by 100.0 (not 100) to force decimal division in integer DBs
```

Q2. DATABASE SCENARIO: You are building an interactivity feature — a drill-down from monthly revenue to weekly breakdown. The dashboard_sales table has: sale_id, sale_date, region, product_category, revenue. Write two SQL queries: Query 1 returns the monthly revenue summary. Query 2 accepts a month parameter and returns weekly revenue for that month (as a drill-down result).

Answer

This is the SQL backbone of a drill-down interaction — the first query populates the summary chart, and clicking a bar triggers the second query with the selected month as a parameter. Understanding this connection is key to building interactive dashboards.

Query 1 — Monthly Revenue Summary (Summary Layer):

```
-- Query 1: Monthly Revenue Summary
SELECT
 DATE_FORMAT(sale_date, '%Y-%m') AS month,
 COUNT(sale_id) AS total_transactions,
```

```
 ROUND(SUM(revenue), 2) AS total_revenue
FROM
 dashboard_sales
GROUP BY
 DATE_FORMAT(sale_date, '%Y-%m')
ORDER BY
 month ASC;
```

Query 2 — Weekly Drill-Down (accepts month parameter e.g. '2024-03'):

```
-- Query 2: Weekly Revenue Drill-Down for selected month
-- Replace '2024-03' with the dashboard parameter value
SELECT
 WEEK(sale_date, 1) AS week_number,
 MIN(sale_date) AS week_start,
 MAX(sale_date) AS week_end,
 COUNT(sale_id) AS transactions,
 ROUND(SUM(revenue), 2) AS weekly_revenue
FROM
 dashboard_sales
WHERE
 DATE_FORMAT(sale_date, '%Y-%m') = '2024-03' -- Passed from dashboard click
GROUP BY
 WEEK(sale_date, 1)
ORDER BY
 week_number ASC;

-- Drill-down hierarchy: Monthly → Weekly → Daily → Transaction-level
```

Q3. DATABASE SCENARIO: To prevent request bloat, you are asked to build a "dashboard usage analytics" table to track which charts are actually being viewed. Design the schema for a chart_view_log table and write a SQL query that identifies "zombie charts" — charts that have had zero views in the last 30 days — so they can be removed from the dashboard.

Answer

This is a meta-analytics problem — using data to make decisions ABOUT the dashboard itself. Finding zombie charts is the evidence-based approach to fighting request bloat without needing subjective debates with stakeholders.

Schema Design:

```
-- Table 1: Registry of all charts on all dashboards
CREATE TABLE dashboard_charts (
 chart_id INT PRIMARY KEY AUTO_INCREMENT,
 dashboard_name VARCHAR(100) NOT NULL,
 chart_name VARCHAR(150) NOT NULL,
 chart_type VARCHAR(50), -- bar, line, pie, kpi_card, etc.
 added_date DATE NOT NULL,
 added_by VARCHAR(100)
);
```

```
-- Table 2: Log every chart view event
CREATE TABLE chart_view_log (
 log_id BIGINT PRIMARY KEY AUTO_INCREMENT,
 chart_id INT NOT NULL,
 user_id VARCHAR(100),
 view_timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
 session_id VARCHAR(100),
 FOREIGN KEY (chart_id) REFERENCES dashboard_charts(chart_id)
);
```

Zombie Chart Detection Query:

```
-- Identify zombie charts: 0 views in last 30 days
SELECT
 dc.chart_id,
 dc.dashboard_name,
 dc.chart_name,
 dc.chart_type,
 dc.added_date,
 COALESCE(MAX(cvl.view_timestamp), 'Never Viewed') AS last_viewed,
 COUNT(cvl.log_id) AS total_views_all_time,
 'ZOMBIE — Recommend Removal' AS action_flag
FROM
 dashboard_charts dc
LEFT JOIN
 chart_view_log cvl
 ON dc.chart_id = cvl.chart_id
 AND cvl.view_timestamp >= DATE_SUB(CURDATE(), INTERVAL 30 DAY)
GROUP BY
 dc.chart_id, dc.dashboard_name, dc.chart_name, dc.chart_type, dc.added_date
HAVING
 COUNT(cvl.log_id) = 0 -- Zero views in last 30 days
ORDER BY
 dc.added_date ASC; -- Oldest zombie charts first

-- LEFT JOIN ensures charts with NO views at all still appear
-- HAVING filters for the 30-day window zero-view condition
-- This output feeds directly into the "Evidence approach" for chart removal
```

Quick Reference: Topic Summary

Topic	Core Takeaway	Key Term
1.1 KPIs	Use BOTH lead (predictive) and lag (resultant) indicators	Lead vs. Lag
1.2 Five-Second Rule	Critical insight must be grasped in 5 seconds — use layout hierarchy	Information Hierarchy

1.3 Gestalt Principles	Group (Proximity), style (Similarity), section (Enclosure)	Visual Perception
1.4 Color Theory	Color = meaning not decoration. Traffic lights, max 5-7 colors	Categorical/Sequential/Diverging
1.5 Chart Selection	Match chart type to: Comparison, Trend, Composition, Correlation	Chart Category Framework
1.6 User Stories	Design for a specific person, not "everyone". Know WHO before WHAT	As a [user] I want...
1.7 Interactivity	Drill-downs for depth, tooltips for context, page nav for audiences	Drill-Down / Tooltip / Slicer
1.8 Request Bloat	Every chart must pass Decision + Audience + Frequency tests	HiPPO Effect / 3-Filter Test